

How Far Off the Recorded Path Can a Gaussian-Splat Driving Simulator Be Trusted?

CS 566 Project Proposal (one document per team) · Kyle Sha (ID 9085525476) · Allen Xia (ID 9085517234) · 29 Sep 2026

Main papers: [1] **Difix3D+** (CVPR 2025 Oral), *re-implemented* · [2] **FaithFusion** (CVPR 2026), *compared* · [3] **HUGSIM** (TPAMI 2026), *simulator for evaluation*

1. Problem Statement

Driving software is increasingly evaluated in simulators reconstructed from real driving logs: the recorded video becomes a **3D Gaussian splat** [13], a scene of millions of small, semi-transparent coloured ellipsoids that can be rendered from any camera position, so the simulated vehicle can leave the recorded trajectory. However, **a reconstruction is reliable only near that trajectory**; after a lane change or with a higher camera (e.g., a truck), rendered images degrade, and a driving system tested on them may behave differently than in reality. This project addresses two questions: **how far from the recorded trajectory a splat-based simulator remains trustworthy, and whether current generative repair methods ("fixers") restore trustworthiness or merely improve visual realism.** We define **trust** as the agreement between the outputs of a lane detector and a vehicle detector on a rendered image and on the **real** image captured at the same pose.

2. Significance and Motivation

Simulation is a principal means of validating driving software at scale; if off-trajectory renders are unreliable, a simulator may accept unsafe behaviour or reject safe behaviour. Both authors are members of Wisconsin Autonomous, a student autonomous-vehicle team, where we develop and test driving software in conventional simulators (MATLAB/Simulink, RoadRunner, CARLA). Their hand-built 3D assets differ visibly from real camera imagery, which limits how far results transfer to the vehicle. Reconstruction-based simulators promise to close this gap, but only if renders stay faithful off the recorded trajectory, and a generative fixer can make an image more convincing while making it less accurate. Understanding this trade-off motivates the project, which draws on camera geometry, 3D reconstruction, image restoration, and CNN-based perception.

3. State of the Art

- **Splat-based simulators:** HUGSIM [3] executes driving policies in reconstructed scenes; the EUVS benchmark [7] shows that current methods degrade under large viewpoint changes.
- **Diffusion-based fixers:** Difix3D+ [1] repairs off-trajectory renders with a single-step diffusion model and adds the repaired views back as training images for the 3D scene, reporting roughly half the FID (a realism score; lower is better) of prior methods. NVIDIA Fixer [4] applies the same approach to driving data, and FaithFusion [2] restricts which pixels a fixer may overwrite, using a per-pixel map of how well the reconstruction already supports each pixel.
- **Evaluation practice:** these methods are evaluated primarily with image-realism metrics (FID, LPIPS) or with shifted cameras for which no real image exists (PRG-Fusion [6]). Some studies do use offset analysis or real off-trajectory images [9, 10, 11], but mainly with image-similarity metrics; we found few comparisons of fixers against real off-trajectory images that also count invented content.

4. Proposed Approach

The work is organised in three tiers. The **core** study relies only on public data, released code and weights, and standard training, and forms a complete project on its own; **extensions** and **stretch goals** are added in order as time permits, and each adds rows or axes to the same evaluation rather than changing it.

- **Core: fixers against real off-trajectory images.** Using Para-Lane [8], which scans the same roads in three parallel lanes (approximately 3 m and 6–7.7 m apart), we reconstruct a splat from one lane with gsplat [13] and render it at the recorded poses of the neighbouring lanes, for which real images exist. We compare the raw render, the released Difix [1] and NVIDIA Fixer [4] with our **re-implementation of the Difix training procedure**: as in the paper, the fixer is fine-tuned from the pretrained SD-Turbo diffusion model, with training code we write from the paper and

validate against NVIDIA's official code and weights. It is trained on driving data, using training pairs built from neighbouring lanes, each pairing a render moved into lane B with the real lane-B image (cf. ReCamDriving [12]). All methods are scored for image similarity, lane and vehicle agreement, and **hallucination** (invented or erased lane markings and spurious vehicles).

- **Extensions:** (a) a **smaller fixer** trained from scratch on the same pairs (a standard 15M-parameter U-Net), to see how much of Difix's behaviour a much smaller model retains; (b) the **road-surface prior** from ViSE [15], re-implemented because no code is released, which fits a smooth road surface to LiDAR and involves no image generation; (c) the **progressive 3D update** of Difix3D+ [1], which adds repaired views back as training images for the 3D scene; and (d) **more test roads** and the **heading** axis.
- **Stretch goals:** (e) **camera height**, using MV2 [9], in which a car and a scooter record the same road at different camera heights; (f) a **cross-dataset evaluation** on US city scenes (EUVS benchmark [7], built on the MARS dataset) and Indian roads (MV2 [9]); (g) **FaithFusion** [2] as an additional comparison method; and (h) **closed-loop evaluation** in the HUGSIM simulator [3].

5. Open Questions and Expected Findings

Fixers are diffusion models trained to produce realistic images, so they may add plausible detail where the true scene is unknown, and realism metrics alone would not reveal this. The study therefore asks:

- How quickly does agreement with the real image fall as the camera moves sideways (core), turns (extension), or changes height (stretch)?
- Do fixers improve lane and vehicle agreement, or mainly image realism, and do they introduce spurious lane markings or vehicles?
- Does training a fixer on real neighbouring-lane pairs make it more faithful (core), and how much of the effect is retained by a much smaller fixer or by a purely geometric road prior (extensions)?
- Do fixers trained on one dataset help on another (stretch)?

We anticipate that the answers will depend on the scene and on the type of deviation; for example, fixers may improve realism more than lane accuracy, and fixers fine-tuned on one city may transfer only partially to others.

6. Evaluation Plan and Expected Results

- **Deviation axes:** lateral offset (core; real images from adjacent lanes at ≈ 3 m and 6–7.7 m); heading (extension; a pure rotation lets the real image be warped exactly to the new view, so only pixels the original camera saw are scored); and camera height (stretch; real car and scooter views from MV2, or, as a fallback, the real image re-projected onto the road plane at the new height).
- **Metrics:** image similarity to the real image (PSNR and LPIPS, standard pixel-level and perceptual similarity scores); **lane-marking precision and recall** (the fraction of painted-marking pixels in the render that match real markings, and the fraction of real markings reproduced, i.e. invented versus erased markings); **lane agreement**, the F1 score between lanes detected by CLNet [14] on the render and on the real image, and the analogous vehicle agreement with YOLO11, following ReconDreamer [5]; **spurious vehicles**, i.e. vehicles detected in the render with no counterpart in the real image; and, as a stretch goal, closed-loop evaluation, in which a released driving policy (LTF) drives inside the HUGSIM simulator [3] and is scored on safety and progress.
- **Expected results.** *Core:* Figure 1, trust versus lateral offset; Table 1, hallucination statistics for each method; Table 2, our Difix re-implementation compared with NVIDIA's official code; and side-by-side videos with hallucinated content annotated. *Extensions* add methods to Figure 1 and Table 1 and a heading curve; *stretch goals* add Figure 2 (trust versus camera height), Table 3 (cross-dataset results), and closed-loop scores.

Components and how each is used:

Component (what it does)	Source	Our use
Difix image fixer [1]: a diffusion model that takes a flawed rendered image and outputs a cleaned-up version	NVIDIA (code, weights)	Re-implemented (our training code) and trained on our pairs
Progressive 3D update [1]: cleaned images are added back as training views, so the 3D scene itself improves off the recorded path	Difix3D+ paper	Re-implemented
FaithFusion [2]: limits which pixels a fixer may change, using a per-pixel confidence map	Released code	Comparison method (run as released)

Component (what it does)	Source	Our use
NVIDIA Fixer [4]: NVIDIA's version of Difix, trained on its own driving data	Released weights	Comparison method (run as released)
Smaller fixer : standard image-to-image U-Net (15M parameters)	Standard architecture	Trained by us from scratch
Road-surface prior [15]: fits a smooth road surface to LiDAR points and flattens the road in the 3D scene; involves no image generation	ViSE paper (no code)	Re-implemented
3D reconstruction [13]: builds each Gaussian-splat scene from the recorded video	gsplat library	Run by us per scene
Detectors and simulator : CLNet lanes [14], YOLO11 vehicles, HUGSIM [3] with the LTF policy	Released, pretrained	Used as-is (evaluation only)

7. Timeline

Dates	Tasks
9/29–10/12	Core : splats of the Para-Lane test roads; evaluation framework (detectors, metrics); raw, released Difix and NVIDIA Fixer results
10/13–10/29	Core : Difix training re-implementation, validation against the official code, training on neighbouring-lane pairs; core figures and tables; mid-term report (10/29)
10/30–11/20	Extensions : smaller fixer, road-surface prior, progressive update, more roads, heading; stretch goals as time permits (camera height, cross-dataset, FaithFusion, closed loop)
11/21–12/10	Final figures and videos; presentation (12/1–12/10); project webpage (12/10)

Risks: the core depends only on public data, released code and weights, and standard fine-tuning, and is scheduled to be complete by the mid-term report, so the project yields a full result even if extensions or stretch goals are not reached. Results may rest on few test roads (we will use as many scenes as time permits and report per-scene variance). Closed-loop evaluation would use one released driving policy (LTF), because the other released policies require outdated software dependencies.

References

1. Difix3D+, CVPR 2025 · arxiv.org/abs/2503.01774 · github.com/nv-tlabs/Difix3D
2. FaithFusion, CVPR 2026 · arxiv.org/abs/2511.21113 · github.com/wangyuanbiubiubiu/FaithFusion
3. HUGSIM, TPAMI 2026 · arxiv.org/abs/2412.01718 · github.com/hy Zhou404/HUGSIM
4. NVIDIA Fixer, 2025 · github.com/nv-tlabs/Fixer
5. ReconDreamer, CVPR 2025 · arxiv.org/abs/2411.19548
6. PRG-Fusion, 2026 · arxiv.org/abs/2609.06948
7. EUVS benchmark, ICCV 2025 · arxiv.org/abs/2412.05256
8. Para-Lane dataset, 3DV 2025 · arxiv.org/abs/2502.15635
9. MV2 dataset, ECCV 2026 · arxiv.org/abs/2608.12442
10. Concept to Capability, SafeComp 2026 · arxiv.org/abs/2605.01995
11. SetDiff, 2026 · arxiv.org/abs/2601.07540
12. ReCamDriving, ECCV 2026 · arxiv.org/abs/2512.03621
13. 3D Gaussian Splatting, SIGGRAPH 2023 · arxiv.org/abs/2308.04079
14. CLNet, CVPR 2022 · arxiv.org/abs/2203.10350
15. ViSE, ICCV 2025 · arxiv.org/abs/2510.18341